Anil Neerukonda Institute of Technology & Sciences
(Autonomous)

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	Semester:	1st sem
Name of the Student:	K. Lokesh	Roll Number:	A24126532148
Name of the Lab:	Full Stack Lab	Timings:	9:40 - 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4 Marks)	Results (5 Marks)	Viva-Voce (2 Marks)			
1/7	1	4	4	5	1	14	10	
8/7	2	4	4	4	2	14	10	
15/7	3	3	4	5	2	14	10	
22/7	4	4	4	5	1	14	10	
29/7	5	4	4	4	2	14	10	
5/8	6	4	4	4	2	14	10	
12/8	7	4	3	4	2	13	10	
16/9	8	4	4	4	1	13	10	
23/9	9	4	3	4	2	13	10	
30/9	10	4	4	4	1	13	10	

Name of the Faculty: ?

Faculty Signature:

INDEX

S.No.	ANITS Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1	01/07	Introduction to Basic HTML	CO1	14	
2	08/07	Design HTML using audio	CO1	14	
3	15/07	Design HTML using video	CO1	14	
4	22/07	Program using JS (arrays)	CO1	14	
5	29/07	Develop Dynamic WebPage	CO1	14	
6	05/08	Implement modules in node JS	CO1	14	
7	12/08	Mini application using express & Node.js	CO1	13	
8	19/08	Code Tantra (Form)	CO1	13	
9	02/09	Code Tantra (Portfolio)	CO1	13	
10	09/09	Code Tantra (simple calculator)	CO1	13	
11	16/09	Functional & class component	CO2	13	
12	23/09	HTTP's using JSON server	CO2	13	
13	30/09	Mongo DB	CO3	13	

01-07-26.

Introduction to HTML and CSS?

→ HTML (Hyper text Markup Language) is the standard markup language used to create and structure content on web Pages. It defines the basic structure of a webPage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by Web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading Style Sheets) :- is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices CSS can be applied using inline, internal, or external stylesheets.

→ HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized attractive, and user-friendly web interfaces.

Difference between MERN stack and FULL stack :

→ MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express-js, React-js, Node-js. MongoDB is used for database management, Express-js handles the backend framework and API's. React-js is used to develop frontend user interface, and Node-js provides the server-side runtime environment.

→ Full-stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

8/07/2026 :-

HTML components :-

① HTML Tags – Define the structure and elements of a webPage.

② Headings – Used to display titles and headings using <h1> to <h6>

③ Paragraphs – Used to display text using the <P> tag.

④ Links – Used to navigate between webpages using <a> tag.

⑤ Images – Used to display images using <image> tag

⑥ Lists – Used to create ordered and unordered lists using <ol>, <ul>, <li>.

⑦ Tables – Used to organize data in rows and columns.

⑧ Forms – Used to collect user input using elements such as text boxes, buttons and checkboxes.

⑨ Div and Span – Used to group and organize webpage content.

⑩ Semantic Elements – Elements such as <header>, <nav>, <section>, <article>, <footer>, provide meaningful webpage structure.

CSS components :-

① Selectors - select HTML elements to apply styles

② Properties - Define the visual characteristic of elements.

③ Values - Specify the setting of a CSS property

④ Colors - Define text, background, border colors.

⑤ Fonts - control font family, size, style and weight.

⑥ Backgrounds - set background colors and images.

⑦ Borders - Define the border style, width and radius.

⑧ Margin - controls the space outside an element.

⑨ Padding - Controls the space inside element.

⑩ Layout - controls the arrangement of elements using Flexbox, grid and positioning.

HTML vs HTML5

HTML	HTML5
Older version of HTML	Latest major version of HTML.
Limited Multimedia support	Supports audio and video.
Uses external Plugins for multimedia	No external Plugs needed.
Has fewer semantic tags	Provides semantic tags.
No <canvas> element	support <canvas>
Limited web storage.	Supports Local storage.
Complex DOCTYPE declaration	simple <!DOCTYPE html>.

CSS vs CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic Styling features	Provides advanced styling features
Limited animations	Supports animations and transitions.
no flexible layout systems	Supports flexbox and Grid.
Limited visual effects	Supports shadows, gradients, effects.
No media queries	supports media queries.
Uses a single specification	Divided into different modules.

15-07-26

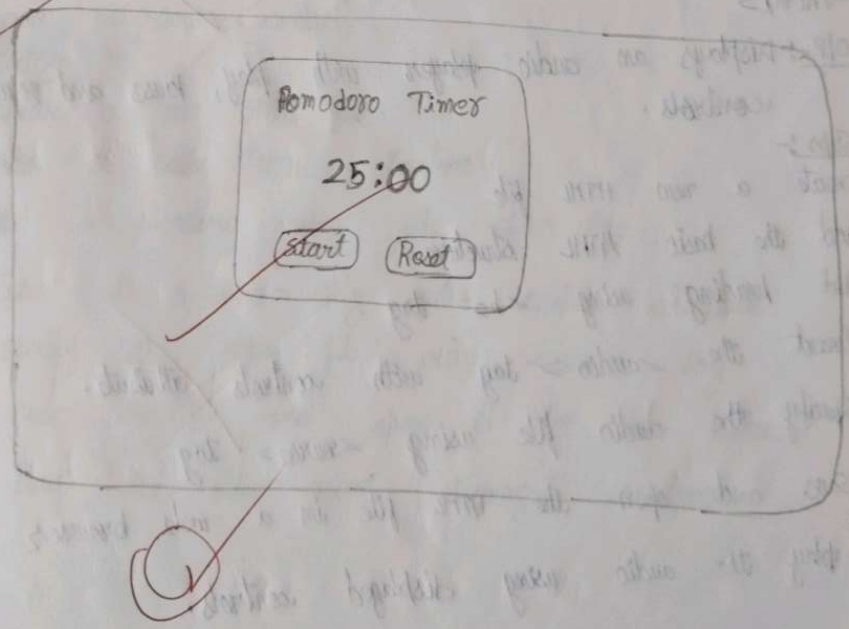
Design a static webpage using HTML and CSS.

Pomodoro Timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            background : #555555;
            text-align : center;
            Padding : 50 PX;
        }
        .timer {
            Width : 300PX;
            margin : auto;
            Padding : 30 PX;
            background : white;
            border-radius : 15PX;
            box-shadow : 0 0 10PX gray;
        }
        h1 {
            color : #074c3c;
        }
        .time {
            font-size : 50 PX;
            margin : 20PX;
        }
        button {
            Padding : 10Px 20PX;
            margin : 5Px;
            border : none;
```

```
        border-radius : 5px;
        background : #e74c3c;
        color : white;
    }
  </style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer</h1>
        <div class = "time" > 25:00 </div>
        <button > start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

O/P:



22/07/26.

1) Design an HTML code using audio tags.

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favourite Song </h2>
    <audio controls>
        <source src = "song.mp3" type = "audio/mpeg">
        Your browser does not support the audio tag
    </audio>
</body>
</html>
```

O/P :- Displays an audio player with play, pause and volume controls.

Steps :-

Create a new HTML file
Add the basic HTML structure.
Add heading using <h2> tag.
Insert the <audio> tag with controls attribute.
Specify the audio file using <source> tag
Save and open the HTML file in a web browser
Play the audio using displayed controls.

(i) Design a HTML code using Video tags :-

code :

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My Video </h2>
    <Video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P :- Displays a video player with play, Pause, volume, and Full screen controls.

Steps :-

Create a new HTML file.
Add a basic HTML structure.
Add a heading using the <h2> tag.
Insert the <video> tag with controls attribute
specify the video file using <source> tag
Save the file and open it in web browser
Play the video using displayed controls.

Write a program using JavaScript (arrays, functions)

Code:

```
Let numbers = [12, 45, 8, 23, 56, 14, 34, 67, 90, 11];

Function displayArray (arr) {
    console.log ("Array Elements:");
    for (let i=0; i<arr.length; i++) {
        console.log (arr[i]);
    }
}

function findSum (arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}
```

Experiment - 4

④ Develop dynamic webpage using java script.

Program :-

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body {
            text-align : center;
            font-family : Arial;
            margin-top : 100 Px;
        }
        button {
            Padding : 10Px 20Px;
            font-size : 16Px;
            cursor : Pointer
        }
    </style>
</head>
<body>
    <h1 id = "heading"> Welcome to My Webpage
    </h1>
```

```
<p id = "message"> click the button to change
                  this webpage. </p>
<button onclick = "changePage()"> click Me </button>
<script>
    function changePage(){
    document.getElementById("heading").
        innerHTML = "Welcome to javascript!";
    document.getElementById("message").
        innerHTML = "The webpage has been
     changed dynamically.";
    document.body.style.backgroundcolor = "lightblue";
        }
    </script>
</body>
</html>
```

Output :-

Welcome to My WebPage

click the button to change the Webpage

[click Me]

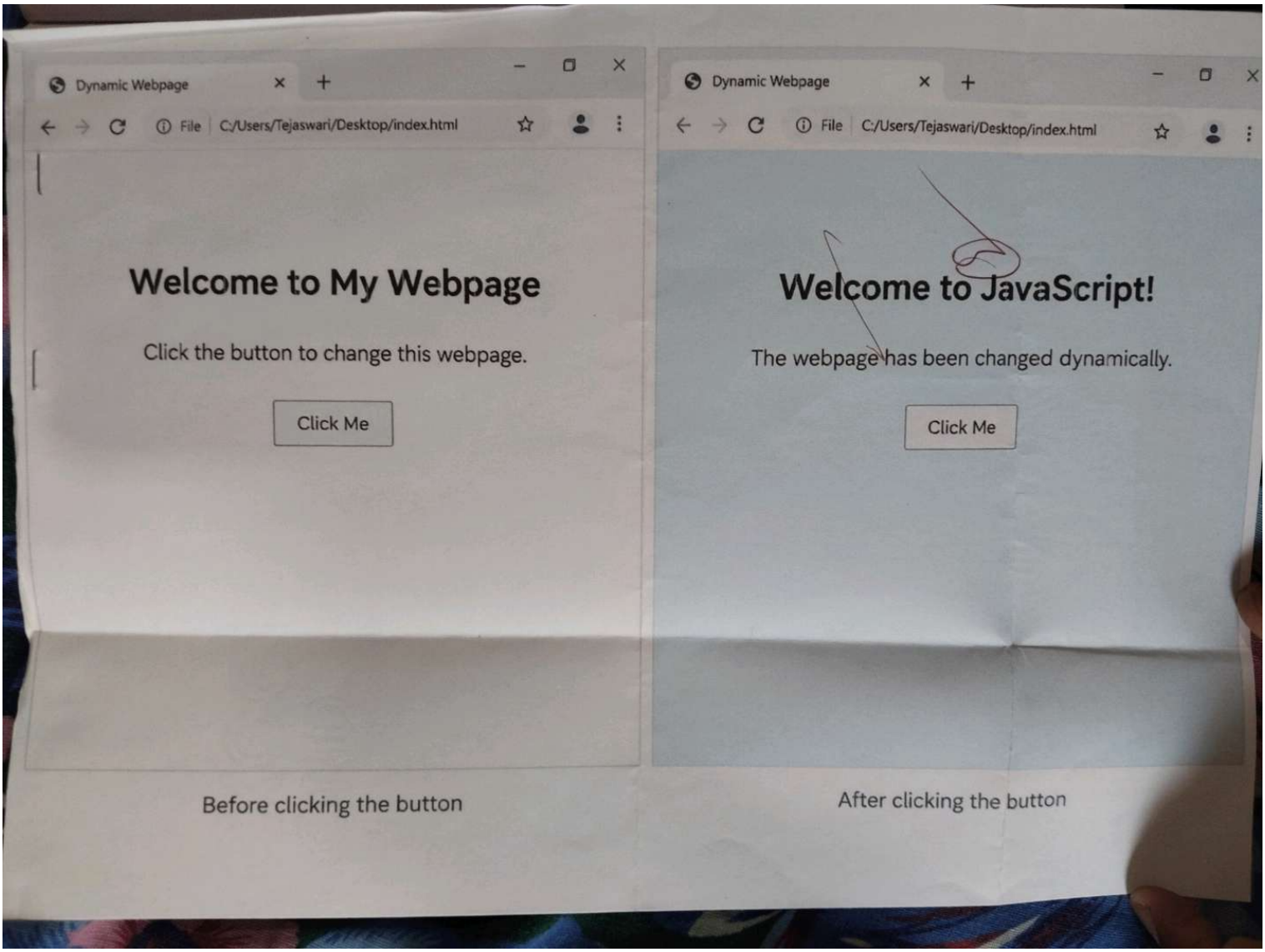
- When the user clicks "click Me", the javascript function changePage() is called, then the heading dynamically changes and background also changes.

After clicking the button :

Welcome to JavaScript!

The Webpage has been changed dynamically

[click Me]



Before clicking the button

After clicking the button

Experiment - 05

⑤ Implement Modules in Node JS.

- In Node.js, you can implement modules using two distinct systems: commonJS (CJS), which is traditional default, and ESModules (ESM), the modern javascript standard.

1. The CommonJs Method (default):

CommonJs uses module.exports to share code and require() to import it. This system loads modules synchronously.

a. Create the module (math.js):

```
Const add = (a,b) => a+b;
const subtract = (a, b) => a-b;
module.exports = {
        add,
        subtract
};
```

b. Use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

Output: 15
5

2. The ES modules Method (modern standard):-

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js):

```
export const multiply = (a, b) => a * b
export const divide = (a, b) => a / b.
const calculatorName = "Supercalc";
export default calculatorName;
```

b. Use the module (main.js):-

```
import calculatorName, {multiply, divide}
                from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

Output:- Supercalc

8

2.

③ Using Built-in Core modules :-

Node.js also comes equipped with dozens of built-in core modules (like fs for file systems or path for file routing) that don't need installation. We will load them by module names :

```
// CommonJs style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

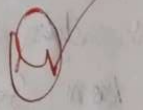
Here are some of the most commonly used ones :

1. fs = file system operations.
2. http - HTTP server and client
3. Path - File Path utilities.
4. Os - operating system utilities
5. events - Event handling
6. util - utility functions
7. Stream - Stream handling
8. crypto - cryptographic functions.
9. Url - URL Parsing
10. querystring - URL query string handling.

Example :- simple HTTP server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

output :- Hello world!



Experiment -06

⑥ Develop mini application using express and NodeJs

- Simple mini application using node Js + Express Js.

Student Management system

1. Add a student
2. View all students
3. delete a student
4. Uses Express.js routes
5. simple HTML interface.

- Project Structure :-

```
student-app/
├── server.js
└── Public
    └── index.html
```

Server.js

```
Const express = require ("express");
Const app = express ();
const PORT = 3000;
let students = [];
app.use (express.json());
app.use (express.static ("Public"));
app.get ("/students", (req, res) => {
        res.json (students);
});
```

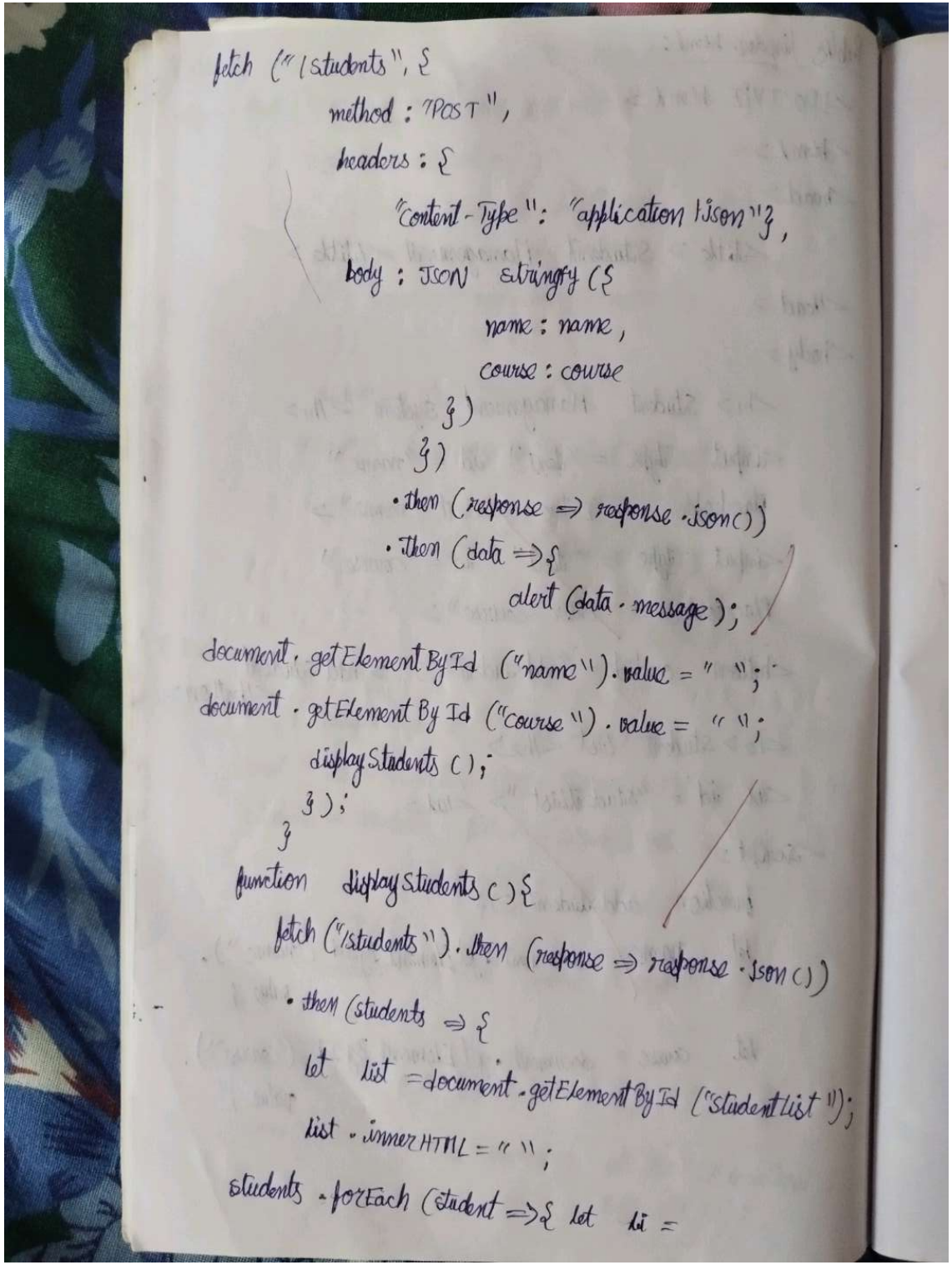
```
// Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "Student added
                         successfully",
               student: student });
});

// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id);
    res.json({
        message: "student deleted Successfully" });

// Start server
app.listen(PORT, () => {
    console.log('Server running at http://localhost:
                ${PORT}');
});
```

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type = "text" id = "name"
    Placeholder = "Enter student name" >
    <input type = "text" id = "course"
    Placeholder = "Enter course" >
    <button onclick = "addStudent()" > Add Student
    </button>
    <h2> Student List </h2>
    <ul id = "studentList" > </ul>
<script>
    function addStudent(){
    let name = document.getElementById("name").
                value ;
    let course = document.getElementById("course").
                value ;
```



```
fetch ("/students", {
        method : "POST",
        headers : {
            "Content-Type": "application/json"},
        body : JSON.stringify ({
                name : name,
                course : course
            })
        })
    .then (response => response.json())
    .then (data =>{
            alert (data.message);
document.getElementById ("name").value = "";
document.getElementById ("course").value = "";
        displayStudents ();
        });
    }
function displayStudents (){
    fetch ("/students").then (response => response.json())
    .then (students => {
    let list =document.getElementById ("studentList");
    list.innerHTML = "";
students.forEach (student =>{ let li =
```

```
document.createElement("li");
    li.innerHTML = student.id + "-" +
                   student.name + "-" +
                   student.course + "<button
onclick = 'deleteStudent(" + student.id + ")'>Delete</button>";
    list1.appendChild(li);
        });
    });
}
function deleteStudent(id){
    fetch("/students/" + id, {method: "DELETE"})
.then(response => response.json())
.then(data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

Output :-

Student Management System

Enter student name

Enter Course

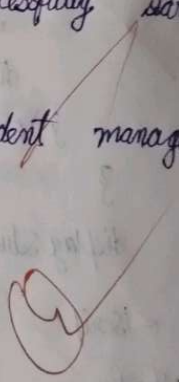
Add Student

Student List :-

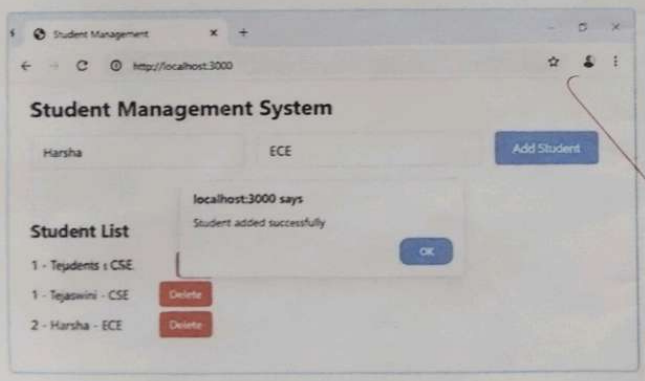
1. Manohar - CSM [delete]
2. Manoj - CSE [delete]

- The Express.js server is successfully started at http://localhost:3000

The browser displays the student management system with fields.



3. After adding a student (alert message)



5. After adding another student (alert message)

6. After

Experiment - 07

(07) Create a Functional component and class component using react.

Aim :- To develop a react application using Functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

(*) Functional component :-

A Functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the webpage dynamically.

Program :-

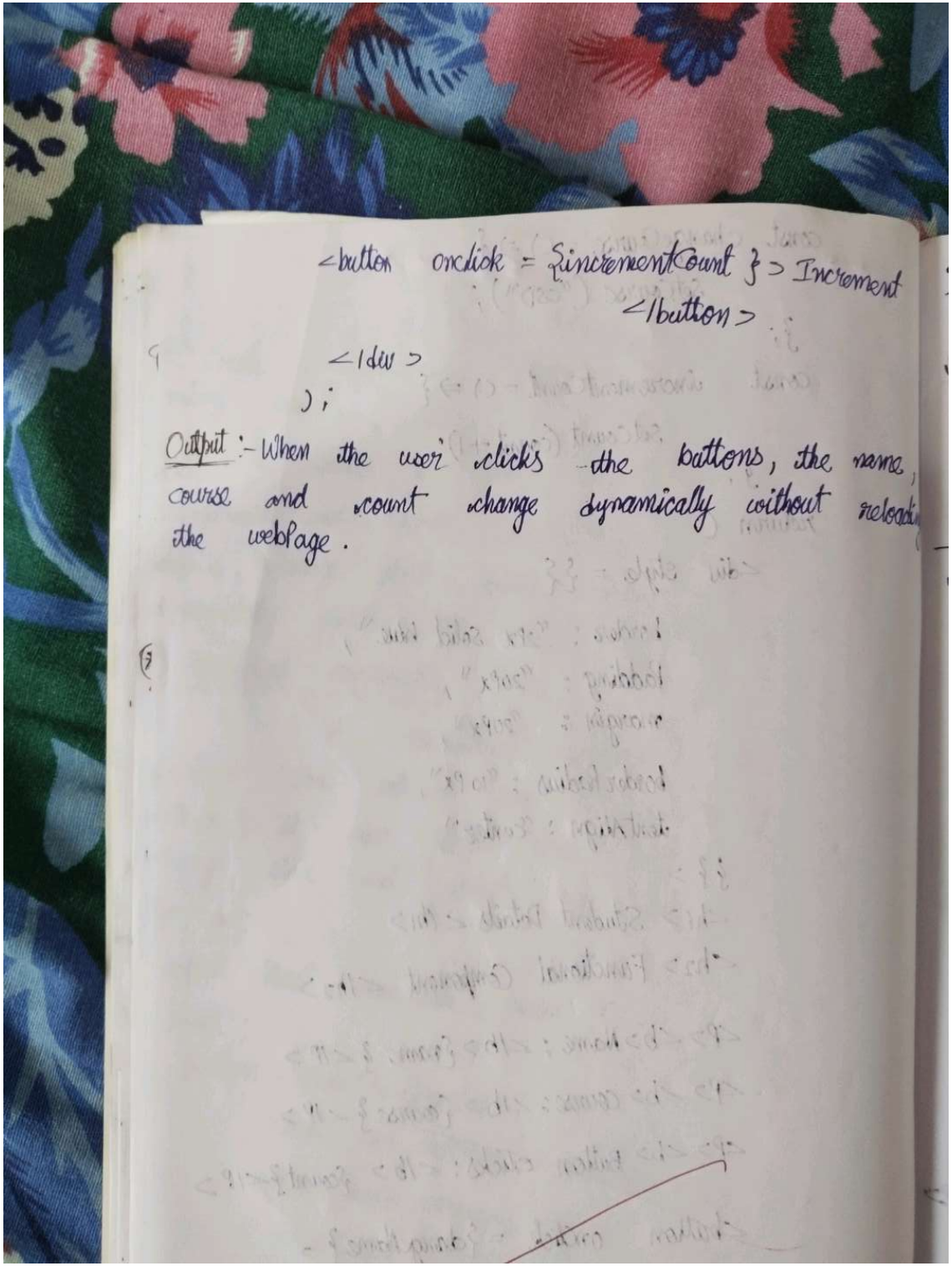
```
import React, {Use state} from
'react';
function studentFunctional () {
    const [name, setName] = UseState("Tejaswari");
    const [course, setCourse] = Use state ("CSM");
    const [count, setCount] = Use State (0);
    const changeName = () => {
        setName ("Teju");
    };
```

```
const changeCourse = () => {
        setCourse ("CSD");
};
const incrementCount = () => {
        setCount (count + 1);
};
return (
    <div style = {{
        border : "2px solid blue",
        Padding : "20px",
        margin : "20px",
        borderRadius : "10 px",
        textAlign : "center"
    }} >
    <h1> Student Details </h1>
    <h2> Functional Component </h2>
    <P> <b> Name : </b> {name} </P>
    <P> <b> course: </b> {course} </P>
    <P> <b> Button clicks: </b> {count} </P>
    <button onClick = {changeName} >
        change Name </button>
    <button onClick = {changeCourse} change
        course </button>
```

```
<button onClick={incrementCount}>Increment
</button>

</div>
);
```

Output :- When the user clicks the buttons, the name, course and count change dynamically without reloading the webpage.



• Class Component :-

A class component is an ESE Javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program :-

```
import React, {Component} from
'react';
class studentclass extends Component {
    constructor (Props) {
        super (Props);
        this.state = {
            name : "Tejasvari",
            course : "computer science",
            count : 0
        };
    }
    changeName = () => {
        this.setstate ({
            name : "Teju"
        });
    };
    changeCourse = () => {
        this.setstate ({
            course : "CSM"
        });
    };
```

```
incrementCount = () => {
    this.setState((prevState) => {
        count: prevState.count + 1,
    });
};
render() {
  return (
    <div style={{
        border: "2px solid green",
        Padding: "20Px",
        margin: "20Px",
        borderRadius: "10Px",
        textAlign: "center"
    }}>
        <h1>Student Details</h1>
        <h2>Class Component</h2>
    <P><b>Name:</b>{this.state.name}</P>
    <P><b>course:</b>{this.state.course}</P>
    <P><b>Button clicks</b>{this.state.count}</P>
<button onClick={this.changeName}>
    changeName</button>
<button onClick={this.changeCourse}>
    changeCourse</button>
```

React App

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Experiment - 8

⑧ Implement HTTP methods using JSON server.

Aim :- To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update and Delete) operations on student records.

Theory :- JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server.
- POST - Adds new data to the server
- PUT - Replaces an existing record.
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server.

a. Create a JSON server :-

Step 1: Initialize the project

Open the VS code terminal and execute the following commands:

```
mkdir JSONserver
cd JSONserver
npm init -y
```

npm install json server.

Step 2: Create db.json

create a file named db.json and add the following student data.

```
{
  "students": [
    {
      "id": "1",
      "name": "Tejaswari",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "chaitrika",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3",
      "name": "Sravani",
      "course": "IT",
      "marks": 88
    }
```

Step 3 :- Start the JSON server.

For the current JSON server version, execute :

npx json-server db.json

The server start at :

http://localhost:3000

The student records can be accessed using http://localhost:3000/students.

2. Implementation of HTTP methods :-

a. GET Method :- The GET method is used to retrieve student records from JSON server.

code :-

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

Output :

B. POST

now

code :-

Output :-

B. POST Method :- The POST method is used to insert a new student record into the server.

Code :-

```
fetch('http://localhost:3000/students', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Rahul',
        course: 'CSE',
        marks: 92
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

```
.catch (error => {
        console.log(error);
});
```

Output :-

C. PUT Method :- The PUT method is used to replace an existing student record with updated data.

Code :-

```
fetch ('http://localhost:3000/students/1', {
    method : 'PUT',
    headers : {
        'content-Type' : 'application/json'
    },
    body : JSON.stringify ({
        name : 'Tejasvari',
        course : 'AIML',
        marks : 95
    })
})
.then (response => response.json ()).
.then (data => {
        console.log (data);
})
```

Output :-

D. PATCH

Updated

Code :- fet

```
• catch (error => {
      console.log(error);
});
```

Output :-

D. PATCH Method :- The PATCH method is used to update only selected fields of an existing student record.

Code :-

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        marks: 96
    })
})
• then(response => response.json())
• then(data => {
```

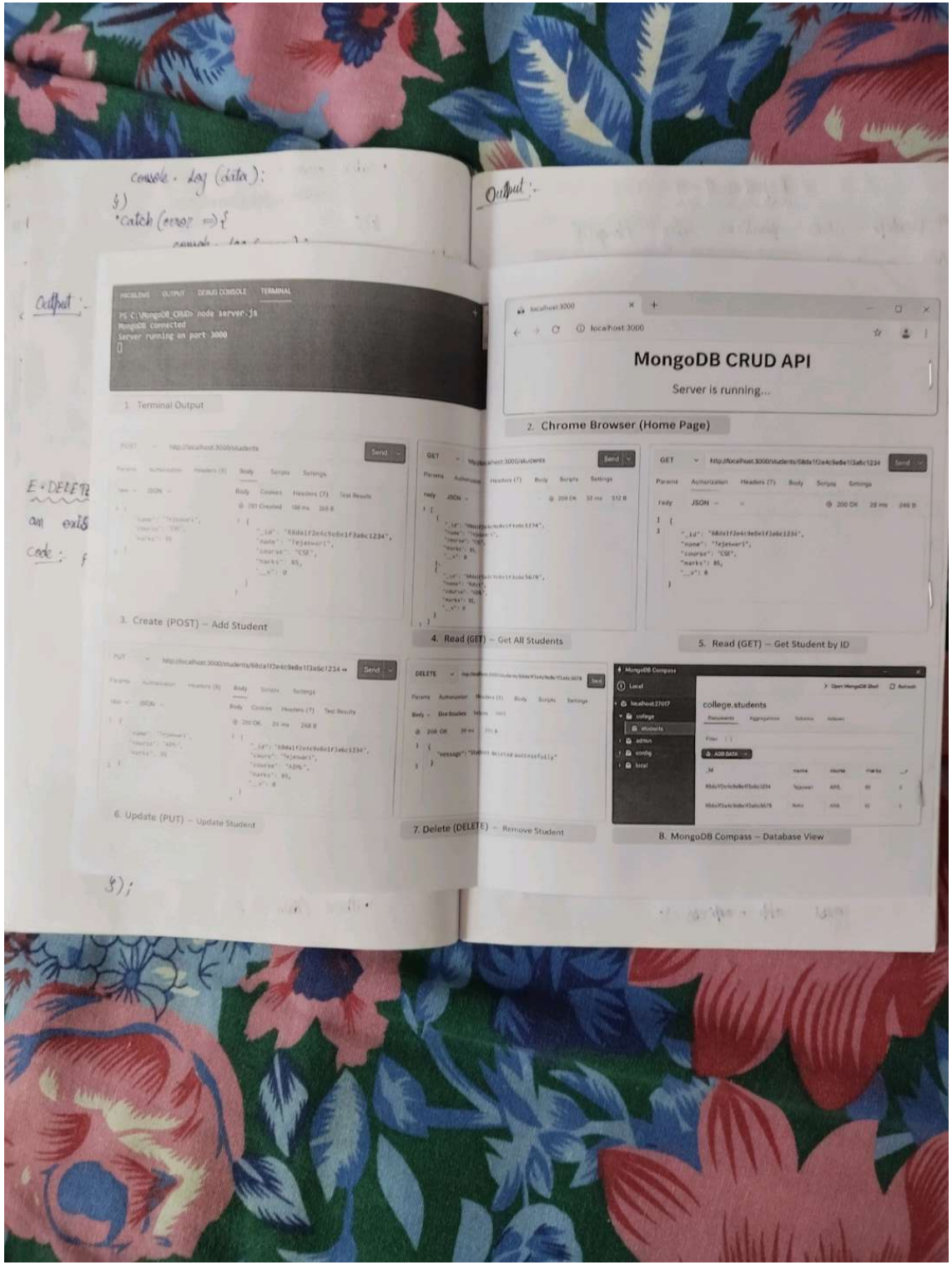
```
    console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

Output :-

E. DELETE Method :- The 'DELETE' method is used to remove an existing student record from the server.

Code :-

```
fetch('http://localhost:3000/students/3', {
    method: 'DELETE'
})
.then(response => {
    if (response.ok) {
        console.log("student deleted successfully");
    }
})
.catch(error => {
    console.log(error)
});
```


console.log(data);
});
.catch(error => {
Output:-
Output:-
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
1. Terminal Output
localhost:3000
MongoDB CRUD API
Server is running...
2. Chrome Browser (Home Page)
3. Create (POST) – Add Student
4. Read (GET) – Get All Students
5. Read (GET) – Get Student by ID
6. Update (PUT) – Update Student
7. Delete (DELETE) – Remove Student
8. MongoDB Compass – Database View
college.students
});

Experiment - 9

⑨ Develop CRUD operations using MongoDB.

Aim :- To develop a web application using Node.js, express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory :- MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and Project setup :-

```
mkdir MongoDB_CRUD
cd MongoDB_CRUD
npm init -y
npm install express mongoose.
```

Program :- server.js

```
const express = require ('express');
const mongoose = require ('mongoose');
const app = express ();
```

```
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/college')
.then(() => console.log("MongoDB connected"))
.catch(err => console.log(err));
// create student
const studentSchema = new
mongoose.schema({
    name: {
        type: String,
        required: true
    },
    course: {
        type: String,
        required: true
    },
    marks: {
        type: Number
        required: true
    }
});
```

```
//create Student model
const Student =
    mongoose.model('Student', studentSchema);
//CREATE: Add
app.post('/students', async (req, res) => {
    try {
        const student = new Student(req.body);
        const savedStudent = await student.save();
        res.status(201).json(savedStudent);
    } catch (error) {
        res.status(400).json({message:
            error.message});
    }
});
//READ: Get all students
app.get('/students', async (req, res) => {
    try {
        const students = await
    student.find();
    res.status(500).json({message: error.message});
    }
});
```

```
//READ:
app.get(
```

```
// READ: get student by ID
app.get('/students/:id', async (req, res) => {
    try {
        const student = await
        Student.findById(req.params.id);
        if (!student){
            return res.status(404).json({
                message: "student not found"
            });
        }
        res.status(200).json(student).
    } catch (error){
        res.status(400).json({message:
            error.message}):
    }
});
// UPDATE
app.put('/students/:id', async (req, res) => {
    try {
        const student = await
Student.findByIdAndUpdate(
            req.params.id,
```

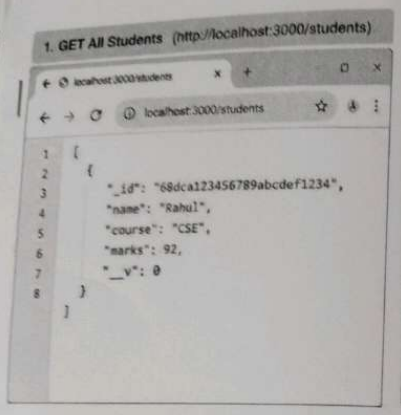
```
        req.body,
        { new: true, runValidators: true }
    );
    if (!student) {
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json(student);
} catch (error) {
    res.status(400).json({ message:
        error.message });
}
});
// DELETE
app.delete('/students/:id', async (req, res) => {
    try {
        const student = await Student.
            findByIdAndDelete(
            req.Params.id
        );
        if (!student) {
            return res.status(404).json({
```

Outputs :-

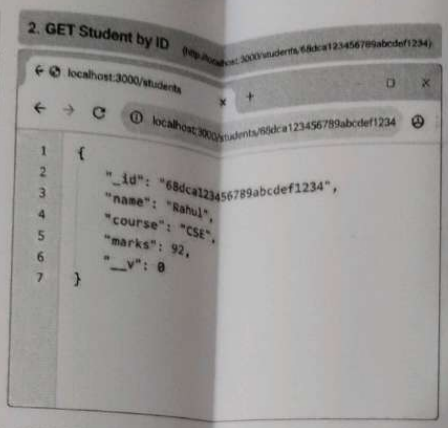
```
            message : "student not found"
        });
    }
    res.status(200).json({
        message : "student deleted successfully"
    });
} catch (error){
    res.status(400).json({message :
                        error.message});
    }
});
```

Outputs :-

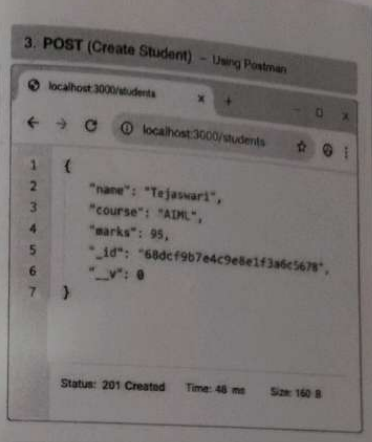
1. GET All Students (http://localhost:3000/students)



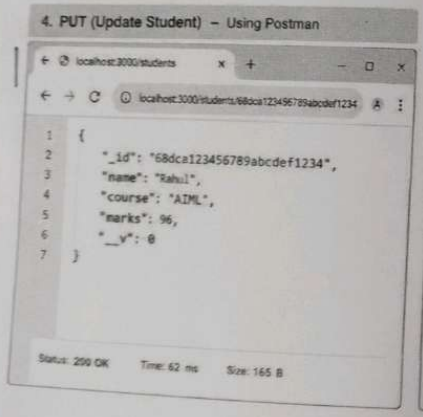
2. GET Student by ID



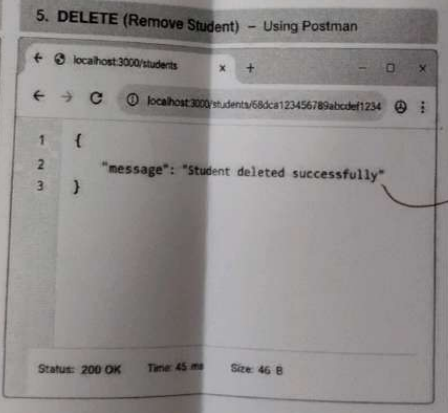
3. POST (Create Student) – Using Postman



4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)

